REPOPILOT

Database Design & Documentation

MongoDB · Data model · Relationships · Indexes · Validation · Retention

From GitHub issue to verified, review-ready pull request.

Detailed proposed database design aligned with RepoPilot PRD v1.0 and API Documentation v1.0. It defines collections, fields, relationships, indexes, lifecycle rules and security controls for the MVP.

Document status: Proposed implementation design. The PRD establishes users, repositories, agentRuns and reports. Supporting collections for approvals, agentEvents and idempotencyKeys are recommended additions.

Property	Value
Database	MongoDB
ODM	Mongoose recommended
Version	1.0 · 31 October 2026
Scope	Public GitHub repositories; Node.js test runner MVP
Related specification	RepoPilot API Documentation v1.0

Contents

- 1. Objectives and principles
- 2. Architecture and relationships
- 3. Collection overview
- 4. users
- 5. repositories
- 6. agentRuns
- 7. reports
- 8. approvals
- 9. agentEvents
- 10. idempotencyKeys
- 11. Index strategy
- 12. Mongoose implementation guidance
- 13. Lifecycle and consistency
- 14. Security and privacy
- 15. Backup and retention
- 16. Example documents
- 17. Implementation checklist
- 18. Assumptions

1. Objectives and principles

MongoDB stores application identity, connected repository metadata, agent execution state, patch metadata, test evidence, approvals and report data. GitHub remains the source of truth for repository content, issues, branches, commits and pull requests. Store only the bounded context and evidence required by RepoPilot; do not mirror entire repositories.

Principle	Design decision
Source of truth	GitHub owns repository and PR state; MongoDB stores snapshots and references.
Ownership	User-owned records carry userId or are accessible only through an owned parent.
Evidence integrity	Persist base commit SHA, patch hash, actual test output metadata and timestamps.
Human approval	Approval binds to exact patch hash and base commit.
Bounded storage	Limit source excerpts, diffs, logs and event payload sizes.
Recoverability	Persist run state and ordered events so clients can recover after disconnects.
Least privilege	Never store raw credentials in ordinary user documents.

2. Architecture and relationships

Use ObjectId references for userId, repositoryId and runId. Embed small historical snapshots (issue title, repository fullName, default branch and base commit SHA) inside runs/reports so reports remain understandable if GitHub metadata changes.

Ownership rule: A repository connection belongs to the user who connected it. Do not assume a globally shared repository record unless implementing a deliberate shared-cache model.

3. Collection overview

Collection	Purpose	Relationship
users	Application identity and GitHub profile.	Root entity.
repositories	Connected public repository metadata.	Many per user.
agentRuns	Workflow execution and results.	Belongs to user and repository.
reports	Patch Evidence Report.	One per run; unique runId.
approvals	Explicit patch publishing approval.	Many per run.
agentEvents	Ordered execution timeline / SSE replay.	Many per run.
idempotencyKeys	Safe retries for mutating API requests.	Scoped to user + route + key.

4. users collection

Purpose: store RepoPilot account identity and GitHub profile. GitHub OAuth tokens must not be stored in plaintext in this collection.

Field	Type	Req.	Description
_id	ObjectId	Yes	Primary key.
githubId	Number	Yes	Unique GitHub numeric ID.
username	String	Yes	GitHub login.
name	String	No	Display name.
avatarUrl	String	No	GitHub avatar URL.

Field	Type	Req.	Description
encryptedCredentialRef	String	No	Reference to encrypted token storage; never raw token.
createdAt / updatedAt	Date	Yes	Mongoose timestamps.

Indexes

```
usersSchema.index({ githubId: 1 }, { unique: true });
usersSchema.index({ username: 1 });
```

Use timestamps: true. GitHub OAuth is the authentication mechanism; password fields are not needed. Store credentials in a dedicated encrypted store or vault with tightly restricted decrypt access.

5. repositories collection

Purpose: represent a public GitHub repository connected by a user. Keep metadata for the dashboard and fetch current source files from GitHub when needed.

Field	Type	Req.	Description
_id	ObjectId	Yes	Primary key.
connectedBy	ObjectId	Yes	Reference to users._id.
githubRepositoryId	Number	Yes	GitHub repository ID.
owner / name	String	Yes	Owner login and repository name.
fullName	String	Yes	Canonical owner/name.
htmlUrl	String	Yes	GitHub URL.
defaultBranch	String	Yes	Default branch snapshot.
primaryLanguage	String	No	Primary language.
description	String	No	Repository description.
visibility	String	Yes	Public only in MVP.
readmeSummary	String	No	Bounded summary.
indexingStatus	String	Yes	pending/running/completed/failed.
indexedTreeSha	String	No	Tree SHA last indexed.
lastIndexedAt	Date	No	Last successful index.
timestamps	Date	Yes	createdAt and updatedAt.

Indexes

```
repositoriesSchema.index(
  { connectedBy: 1, githubRepositoryId: 1 }, { unique: true }
);
repositoriesSchema.index({ connectedBy: 1, updatedAt: -1 });
repositoriesSchema.index({ fullName: 1 });
```

Reconnect should update the existing record. Do not make fullName globally unique because multiple users may connect the same public repository.

6. agentRuns collection

Purpose: central workflow execution record. It captures selected issue context, agent stage, investigation, relevant files, patch metadata, test outcomes, errors and timestamps.

Field	Type	Req.	Description
_id	ObjectId	Yes	Run ID.
userId	ObjectId	Yes	Reference to users.
repositoryId	ObjectId	Yes	Reference to repositories.
issueNumber	Number	Yes	GitHub issue number.
issueSnapshot	Object	Yes	Title, body excerpt, URL, labels, capturedAt.
repositorySnapshot	Object	Yes	Full name, default branch, base commit SHA.
status	String	Yes	queued/running/awaiting_review/completed/failed/cancelled.
currentStage	String	No	Current orchestrator stage.
stageResults	Array	No	Stage status, timestamps, duration and output reference.
investigation	Object	No	Classification, root-cause hypotheses and evidence.
relevantFiles	Array	No	Paths, reasons, blob SHA and retrieval metadata.
generatedPatch	Object	No	Patch hash, diff reference, file summary and validation.

Field	Type	Req.	Description
testResults	Object	No	Commands, status, exit code, output references, duration.
approvalState	String	No	not_requested/pending/approved/invalidated/rejected.
pullRequest	Object	No	PR number, URL, branches and createdAt.
error	Object	No	Stable code, safe message, stage, retryable.
startedAt / completedAt	Date	No	Execution timestamps.
timestamps	Date	Yes	createdAt and updatedAt.

Indexes

```
agentRunsSchema.index({ userId: 1, createdAt: -1 });
agentRunsSchema.index({ repositoryId: 1, issueNumber: 1, createdAt: -1 });
agentRunsSchema.index({ status: 1, updatedAt: 1 });
```

Avoid unbounded diffs, source files and terminal logs inline. Store bounded summaries and references to artifact storage. A capped inline diff is acceptable only with a strict size limit.

7. reports collection

Purpose: produce the Patch Evidence Report. It should be traceable to the run, base commit, patch hash and actual test execution.

Field	Type	Req.	Description
_id	ObjectId	Yes	Report ID.
runId	ObjectId	Yes	Reference to agentRuns; unique.
userId	ObjectId	Yes	Owner.
repository	Object	Yes	Full name, URL and repository ID snapshot.
issue	Object	Yes	Number, title and URL snapshot.
investigationSummary	String	No	Findings with uncertainty.
inspectedFiles	Array	No	Paths and retrieval metadata.
filesChanged	Array	No	Path, status, additions, deletions.
patchSha256	String	No	Hash of exact proposed patch.
patchArtifactRef	String	No	Storage reference for full diff.
testEvidence	Object	No	Status, commands, exit code, duration, bounded output.
pullRequest	Object	No	URL, number, state if created.
timestamps	Date	Yes	Created and updated timestamps.

Indexes

```
reportsSchema.index({ runId: 1 }, { unique: true });
reportsSchema.index({ userId: 1, createdAt: -1 });
```

If no test ran, status must be not_run. Failed commands retain failed status and exit code. Missing output must never be converted into a passing result.

8. approvals collection

Purpose: audit the user's explicit approval. Approval is bound to the exact patch and base commit; if either changes, the approval becomes invalid.

Field	Type	Req.	Description
_id	ObjectId	Yes	Approval ID.
runId	ObjectId	Yes	Reference to agentRuns.
userId	ObjectId	Yes	Approving user.
patchSha256	String	Yes	Hash of reviewed patch.
baseCommitSha	String	Yes	Commit used as patch base.
approved	Boolean	Yes	Approval decision.
approvedAt	Date	No	Approval time.
invalidatedAt	Date	No	Invalidation time.
invalidationReason	String	No	Why approval no longer applies.
createdAt	Date	Yes	Audit timestamp.

```
approvalsSchema.index({ runId: 1, createdAt: -1 });
approvalsSchema.index({ runId: 1, patchSha256: 1, baseCommitSha: 1 });
```

Prefer append-only approval history. Do not silently overwrite an earlier approval when a new patch is generated.

9. agentEvents collection

Purpose: ordered, persisted execution events for live SSE updates and recovery after network interruption. Events must reflect real backend transitions.

Field	Type	Req.	Description
_id	ObjectId	Yes	Event ID.
runId	ObjectId	Yes	Reference to agentRuns.
sequence	Number	Yes	Monotonic sequence per run.
eventType	String	Yes	stage.started, stage.completed, test.output, agent.failed, etc.
payload	Object	Yes	Validated, bounded event payload.
timestamp	Date	Yes	Event time.
createdAt	Date	Yes	Persistence time.

```
agentEventsSchema.index({ runId: 1, sequence: 1 }, { unique: true });
agentEventsSchema.index({ runId: 1, timestamp: 1 });
```

Do not persist unlimited terminal output chunks. Store bounded chunks or a reference to a log artifact. TTL cleanup is optional only after retention is decided.

10. idempotencyKeys collection

Purpose: prevent duplicate agent runs, test jobs or pull requests when a client retries after a timeout. Especially important for GitHub write operations.

Field	Type	Req.	Description
_id	ObjectId	Yes	Record ID.
userId	ObjectId	Yes	Authenticated user.
route	String	Yes	Normalized API action.
key	String	Yes	Client Idempotency-Key.
requestHash	String	Yes	Hash of normalized body.
status	String	Yes	processing/completed/failed.
responseSnapshot	Object	No	Safe response for replay.
resourceId	String	No	Run/test/PR ID.
expiresAt	Date	Yes	Retention expiry.

```
idempotencyKeysSchema.index(
  { userId: 1, route: 1, key: 1 }, { unique: true }
);
idempotencyKeysSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 });
```

11. Index strategy

Collection	Index	Purpose
users	{ githubId: 1 } unique	One account per GitHub identity.
repositories	{ connectedBy: 1, githubRepositoryId: 1 } unique	Prevent duplicate user connections.
repositories	{ connectedBy: 1, updatedAt: -1 }	Dashboard listing.
agentRuns	{ userId: 1, createdAt: -1 }	Recent run history.
agentRuns	{ repositoryId: 1, issueNumber: 1 }	Runs for an issue.
agentRuns	{ status: 1, updatedAt: 1 }	Worker recovery scans.
reports	{ runId: 1 } unique	One canonical report per run.
approvals	{ runId: 1, createdAt: -1 }	Approval history.
agentEvents	{ runId: 1, sequence: 1 } unique	Ordered events.
idempotencyKeys	{ userId: 1, route: 1, key: 1 } unique	Retry safety.
idempotencyKeys	{ expiresAt: 1 } TTL	Expire old keys.

Index only fields used by real query patterns. Indexes consume memory and increase write cost. Validate with explain() as the application grows.

12. Mongoose implementation guidance

Use TypeScript types and Mongoose schemas with strict mode, timestamps and explicit enums. Validate incoming API payloads with Zod; Mongoose validation is a second line of defence.

```
const AgentRunSchema = new Schema({
  userId: { type: Schema.Types.ObjectId, ref: "User", required: true },
  repositoryId: { type: Schema.Types.ObjectId, ref: "Repository", required: true },
  issueNumber: { type: Number, required: true, min: 1 },
  status: {
    type: String,
    enum: ["queued", "running", "awaiting_review",
           "completed", "failed", "cancelled"],
    default: "queued",
    required: true
  },
  currentStage: {
    type: String,
    enum: ["issue_analysis", "repository_investigation",
```

```
            "root_cause_analysis", "patch_generation",
            "patch_validation", "test_execution", "pr_preparation"]
    },
    investigation: { type: Schema.Types.Mixed },
    relevantFiles: { type: [Schema.Types.Mixed], default: [] },
    generatedPatch: { type: Schema.Types.Mixed },
    testResults: { type: Schema.Types.Mixed },
    error: { type: Schema.Types.Mixed }
}, { timestamps: true, strict: true });
```

The Mixed fields above are a compact starting example. Replace them with explicit nested schemas for stable production structures such as investigation, patch and test evidence.

13. Lifecycle and consistency

The agentRuns document is the authoritative workflow snapshot. Every stage transition should update the run and append an event. Use MongoDB transactions where supported; otherwise use an idempotent outbox/event pattern so state and events can be reconciled.

Patch consistency: calculate SHA-256 over a canonical patch representation and store the base commit SHA. Approval includes both. If either changes, invalidate prior approval. Before PR creation, recompute the hash and recheck the upstream base.

Report consistency: create/update a report as evidence becomes available. The unique runId index prevents duplicate canonical reports. Use explicit states such as pending, not_run, passed, failed and timed_out.

14. Security and privacy

Risk	Control
Credential exposure	No raw GitHub token in user documents; encrypt token material and restrict decrypt access.
Cross-user access	Store userId and enforce ownership checks in every service query.
Prompt injection	Treat issue, README and source text as untrusted data, not instructions.
Oversized documents	Bound issue excerpts, source snippets, diffs, logs and event payloads.
Sensitive logs	Redact secrets; never persist environment variables or container credentials.
Unauthorized PR	Verify approval, patch hash and base SHA immediately before publishing.
Database exposure	Use TLS, network restrictions and separate least-privilege credentials per environment.

15. Backup and retention

Enable managed backups or scheduled snapshots. Define recovery objectives and test restoration. Use separate database credentials for development, staging and production.

Data	Suggested starting policy
Users and repository links	Retain while account active; support account deletion.
Agent runs and reports	Retain for product history; define export/deletion policy.
Raw test output	Bound size; retain only as long as needed for report/debugging.
Agent events	Keep enough history for report and SSE recovery; optionally TTL old events.
Idempotency keys	Expire after a bounded period such as 24–72 hours.
Credential material	Delete/revoke on disconnect or account deletion; rotate after suspected compromise.

These are proposed policies, not PRD mandates. Ensure deletion covers related reports, events, artifacts and credentials.

16. Example documents

```
// Simplified agentRuns document
{
  "_id": "ObjectId(run)",
  "userId": "ObjectId(user)",
  "repositoryId": "ObjectId(repository)",
  "issueNumber": 128,
  "issueSnapshot": {
    "title": "Fix authentication timeout",
    "htmlUrl": "https://github.com/owner/repo/issues/128"
  },
  "repositorySnapshot": {
    "fullName": "owner/repo",
```

```
    "defaultBranch": "main",
    "baseCommitSha": "4a9bc81f..."
  },
  "status": "awaiting_review",
  "currentStage": "pr_preparation",
  "generatedPatch": { "patchSha256": "sha256:..." },
  "testResults": { "status": "passed", "exitCode": 0 },
  "approvalState": "pending",
  "createdAt": "2026-10-31T09:30:00Z"
}
// Simplified reports document
{
  "runId": "ObjectId(run)",
  "userId": "ObjectId(user)",
  "repository": { "fullName": "owner/repo" },
  "issue": { "number": 128, "title": "Fix authentication timeout" },
  "inspectedFiles": ["src/auth/session.ts", "tests/auth.test.ts"],
  "filesChanged": [
    { "path": "src/auth/session.ts", "status": "modified" }
  ],
  "patchSha256": "sha256:...",
  "testEvidence": {
    "status": "passed",
    "commands": [{ "command": "npm test", "exitCode": 0 }]
  }
}
```

17. Implementation checklist

Task	Definition of done
Type definitions	Types exist for all collections and nested data.
Mongoose schemas	Required fields, enums, timestamps and strict mode.
Indexes	Declared and verified in MongoDB.
Ownership guards	Every repository/run/report query checks ownership.
Zod validation	All API body, path and query inputs validated.
Run state machine	Legal transitions enforced and persisted.
Event persistence	Events ordered, persisted and replayable.
Approval binding	Approval tied to exact patch hash and base commit.
Report persistence	One report per run; truthful test status.
Idempotency	Retries cannot create duplicate PRs.
Integration tests	CRUD, authorization, uniqueness and recovery tested.
Backup/retention	Policies documented and restore tested.

18. Assumptions and decisions

Topic	Decision in this design
Database	MongoDB, as specified in RepoPilot PRD.
ODM	Mongoose recommended for Node.js/TypeScript stack.
Baseline collections	users, repositories, agentRuns and reports from PRD.
Supporting collections	approvals, agentEvents and idempotencyKeys are design additions.
Repository scope	Public GitHub repositories for MVP.
Runner	Node.js test execution first.
Credential storage	Encrypted server-side store; exact vault remains deployment choice.
Artifact storage	Large diffs/logs may use object storage; provider and limits to confirm.
Retention	Proposed starting policies; product owner confirmation required.

Database design definition of done: RepoPilot can persist users, connected repositories, agent runs, real stage events, validated patches, actual test evidence, explicit patch approvals and one auditable report per run. All user-owned records are access-controlled and indexed for expected queries.

End of document · RepoPilot Database Design v1.0